```
"""Faceoff: daily NHL pick'em vs the NHL Analytica model, with share cards.

Run locally:   flask --app app run
Cron (every 5 min on game nights):   flask --app app sync
"""
import os
import re
import sqlite3
import time
import uuid
from datetime import date, datetime, timedelta, timezone
from zoneinfo import ZoneInfo

from flask import Flask, g, jsonify, request, send_from_directory

import model
import nhl

ET = ZoneInfo("America/New_York")
DB_PATH = os.environ.get("FACEOFF_DB", os.path.join(os.path.dirname(__file__), "f
SEASON_START = os.environ.get("FACEOFF_SEASON_START", "2026-10-01")
SYNC_EVERY = 60  # seconds between lazy syncs per date
COOKIE = "fo_uid"
NICK_RE = re.compile(r"^[\w .\-]{2,20}$", re.UNICODE)

app = Flask(__name__, static_folder="static", static_url_path="")

SCHEMA = """
CREATE TABLE IF NOT EXISTS users(
  id TEXT PRIMARY KEY, nickname TEXT UNIQUE COLLATE NOCASE, created INTEGER);
CREATE TABLE IF NOT EXISTS games(
  id INTEGER PRIMARY KEY, date TEXT, start_utc TEXT, home TEXT, away TEXT,
  home_name TEXT, away_name TEXT, p_home REAL, state TEXT,
  home_score INTEGER, away_score INTEGER, period_type TEXT, winner TEXT);
CREATE INDEX IF NOT EXISTS games_date ON games(date);
CREATE TABLE IF NOT EXISTS picks(
  user_id TEXT, game_id INTEGER, team TEXT, created INTEGER, correct INTEGER,
  PRIMARY KEY(user_id, game_id));
CREATE INDEX IF NOT EXISTS picks_game ON picks(game_id);
CREATE TABLE IF NOT EXISTS meta(key TEXT PRIMARY KEY, value TEXT);
"""
```

```
# ---------- db ----------
def db():
    if "db" not in g:
        g.db = sqlite3.connect(DB_PATH, timeout=10)
        g.db.row_factory = sqlite3.Row
        g.db.execute("PRAGMA journal_mode=WAL")
    return g.db


@app.teardown_appcontext
def _close(_exc):
    conn = g.pop("db", None)
    if conn:
        conn.close()


def init_db():
    with app.app_context():
        db().executescript(SCHEMA)
        db().commit()


# ---------- helpers ----------
def today_et():
    return datetime.now(ET).date().isoformat()


def valid_date(s):
    try:
        return date.fromisoformat(s).isoformat()
    except (TypeError, ValueError):
        return None


def started(row):
    if row["state"] not in ("FUT", "PRE"):
        return True
    try:
        start = datetime.fromisoformat(row["start_utc"].replace("Z", "+00:00"))
    except (AttributeError, ValueError):
        return False
    return datetime.now(timezone.utc) >= start


def model_pick(row):
    return row["home"] if row["p_home"] >= 0.5 else row["away"]
```

```
def current_user():
    uid = request.cookies.get(COOKIE)
    if not uid:
        return None
    return db().execute("SELECT * FROM users WHERE id=?", (uid,)).fetchone()


# ---------- sync + grading ----------
def sync_date(d, force=False):
    conn = db()
    key = f"sync:{d}"
    last = conn.execute("SELECT value FROM meta WHERE key=?", (key,)).fetchone()
    if not force and last and time.time() - float(last["value"]) < SYNC_EVERY:
        return
    # claim the slot first so concurrent requests don't all hit the API
    conn.execute("INSERT OR REPLACE INTO meta VALUES(?,?)", (key, str(time.time()
    conn.commit()
    try:
        games = nhl.games_on(d)
    except Exception as e:  # API down: keep serving what we have
        app.logger.warning("schedule fetch failed for %s: %s", d, e)
        return

    table = None
    for gm in games:
        row = conn.execute("SELECT p_home FROM games WHERE id=?", (gm["id"],)).fe
        if row is None:
            if table is None:
                try:
                    table = nhl.standings()
                except Exception as e:
                    app.logger.warning("standings fetch failed: %s", e)
                    table = {}
            p = model.win_prob(gm["home"], gm["away"], table)
            conn.execute(
                "INSERT INTO games(id,date,start_utc,home,away,home_name,away_nam
                "home_score,away_score,period_type) VALUES(?,?,?,?,?,?,?,?,?,?,?,
                (gm["id"], d, gm["start_utc"], gm["home"], gm["away"], gm["home_n
                 gm["away_name"], p, gm["state"], gm["home_score"], gm["away_scor
                 gm["period_type"]))
        else:  # model odds stay frozen once published; only live fields update
            conn.execute(
                "UPDATE games SET start_utc=?, state=?, home_score=?, away_score=
                (gm["start_utc"], gm["state"], gm["home_score"], gm["away_score"]
```

```
                gm["period_type"], gm["id"]))

        if gm["state"] in nhl.FINAL_STATES and gm["home_score"] is not None:
            winner = gm["home"] if gm["home_score"] > gm["away_score"] else gm["a
            conn.execute("UPDATE games SET winner=? WHERE id=?", (winner, gm["id"
            conn.execute(
                "UPDATE picks SET correct=(team=?) WHERE game_id=? AND correct IS
                (winner, gm["id"]))
    conn.commit()


# ---------- stats ----------
def user_stats(uid):
    conn = db()
    rows = conn.execute(
        "SELECT p.team, p.correct, g.* FROM picks p JOIN games g ON g.id=p.game_i
        "WHERE p.user_id=? AND g.date>=? ORDER BY g.start_utc DESC", (uid, SEASON
    graded = [r for r in rows if r["correct"] is not None]
    streak = 0
    for r in graded:
        if r["correct"]:
            streak += 1
        else:
            break
    # record vs model, per night, on the games the user actually picked
    nights = {}
    for r in rows:
        nights.setdefault(r["date"], []).append(r)
    w = l = t = 0
    for picks in nights.values():
        if any(r["correct"] is None for r in picks):
            continue  # night still in progress
        me = sum(r["correct"] for r in picks)
        bot = sum(model_pick(r) == r["winner"] for r in picks)
        w, l, t = w + (me > bot), l + (me < bot), t + (me == bot)
    return {"correct": sum(r["correct"] for r in graded), "graded": len(graded),
            "streak": streak, "vs_model": {"w": w, "l": l, "t": t}}


# ---------- routes ----------
@app.get("/")
def index():
    return send_from_directory(app.static_folder, "index.html")


@app.get("/api/slate")
```

```
def slate():
    d = valid_date(request.args.get("date")) or today_et()
    sync_date(d)
    if d == today_et():  # late West Coast games finish after midnight ET
        sync_date((date.fromisoformat(d) - timedelta(days=1)).isoformat())
    user = current_user()
    rows = db().execute("SELECT * FROM games WHERE date=? ORDER BY start_utc, id"
    mine = {}
    if user:
        mine = {r["game_id"]: r for r in db().execute(
            "SELECT * FROM picks WHERE user_id=? AND game_id IN (SELECT id FROM g
            (user["id"], d))}
    games = []
    for r in rows:
        p = mine.get(r["id"])
        games.append({
            "id": r["id"], "start_utc": r["start_utc"], "state": r["state"],
            "home": r["home"], "away": r["away"], "home_name": r["home_name"], "a
            "p_home": r["p_home"], "home_score": r["home_score"], "away_score": r
            "period_type": r["period_type"], "winner": r["winner"], "locked": sta
            "model_pick": model_pick(r), "my_pick": p["team"] if p else None,
            "correct": None if not p or p["correct"] is None else bool(p["correct
        })
    return jsonify({"date": d, "today": today_et(), "games": games})


@app.get("/api/me")
def me():
    user = current_user()
    if not user:
        return jsonify({"user": None})
    return jsonify({"user": {"nickname": user["nickname"]}, "stats": user_stats(u


@app.post("/api/me")
def set_nickname():
    nick = ((request.get_json(silent=True) or {}).get("nickname") or "").strip()
    if not NICK_RE.match(nick):
        return jsonify({"error": "Use 2-20 letters, numbers, spaces, dots, dashes
    user = current_user()
    conn = db()
    try:
        if user:
            conn.execute("UPDATE users SET nickname=? WHERE id=?", (nick, user["i
            uid = user["id"]
        else:
```

```
            uid = uuid.uuid4().hex
            conn.execute("INSERT INTO users VALUES(?,?,?)", (uid, nick, int(time.
        conn.commit()
    except sqlite3.IntegrityError:
        return jsonify({"error": "That name is taken. Try another."}), 409
    resp = jsonify({"user": {"nickname": nick}, "stats": user_stats(uid)})
    resp.set_cookie(COOKIE, uid, max_age=60 * 60 * 24 * 730, httponly=True,
                    samesite="Lax", secure=request.is_secure)
    return resp


@app.post("/api/pick")
def pick():
    user = current_user()
    if not user:
        return jsonify({"error": "Choose a leaderboard name first."}), 401
    body = request.get_json(silent=True) or {}
    row = db().execute("SELECT * FROM games WHERE id=?", (body.get("game_id"),)).
    if not row:
        return jsonify({"error": "Game not found."}), 404
    if started(row):
        return jsonify({"error": "This game has started. Picks are locked."}), 40
    team = body.get("team")
    if team not in (row["home"], row["away"]):
        return jsonify({"error": "Pick one of the two teams."}), 400
    db().execute(
        "INSERT INTO picks(user_id,game_id,team,created) VALUES(?,?,?,?) "
        "ON CONFLICT(user_id,game_id) DO UPDATE SET team=excluded.team, created=e
        (user["id"], row["id"], team, int(time.time())))
    db().commit()
    return jsonify({"ok": True, "game_id": row["id"], "team": team})


@app.get("/api/leaderboard")
def leaderboard():
    scope = request.args.get("scope", "season")
    if scope == "day":
        d = valid_date(request.args.get("date")) or today_et()
        where, args = "g.date=?", [d]
    else:
        where, args = "g.date>=?", [SEASON_START]
    rows = db().execute(f"""
        SELECT u.nickname, SUM(p.correct) AS correct, COUNT(*) AS graded
        FROM picks p JOIN games g ON g.id=p.game_id JOIN users u ON u.id=p.user_i
        WHERE p.correct IS NOT NULL AND {where}
        GROUP BY u.id ORDER BY correct DESC, (1.0*correct/graded) DESC, u.nicknam
```

```
        LIMIT 50""", args).fetchall()
    games = db().execute(f"SELECT * FROM games g WHERE winner IS NOT NULL AND {wh
    bot = sum(model_pick(r) == r["winner"] for r in games)
    user = current_user()
    return jsonify({
        "scope": scope,
        "rows": [{"nickname": r["nickname"], "correct": r["correct"], "graded": r
        "model": {"correct": bot, "graded": len(games)},
        "me": user["nickname"] if user else None,
    })


@app.cli.command("sync")
def sync_cmd():
    """Refresh yesterday and today (ET) and grade finished games."""
    now = datetime.now(ET).date()
    for d in (now - timedelta(days=1), now):
        sync_date(d.isoformat(), force=True)
        print("synced", d)


init_db()

if __name__ == "__main__":
    app.run(debug=True)
```